

R/Quarto Help Session

DATA607 - Assignment 2B Support Log

R/Quarto Help Session — DATA607 Assignment 2B

Topic: Logistic regression prediction table cleanup, threshold-based confusion matrix (TP/TN/FP/FN), metrics pipeline, and Quarto/RStudio troubleshooting

Q: `round(.pred_female, digits = 2)` — object `'pred_female'` not found

`round()` doesn't work as a dplyr verb on a data frame in a pipe. Use `mutate()`:

```
penguin_model <- penguin_model |>
  mutate(.pred_female = round(.pred_female, digits = 2))
```

For multiple prediction columns:

```
penguin_model <- penguin_model |>
  mutate(across(starts_with(".pred"), \(x) round(x, digits = 2)))
```

Q: How to make a stacked bar chart with percentages

Using `position = "fill"` (auto-computed proportions):

```
ggplot(penguin_model, aes(x = sex, fill = .pred_class)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
```

```
labs(y = "Percentage", x = "Actual Sex", fill = "Predicted Class")
```

Precomputing percentages for labeled bars:

```
penguin_summary <- penguin_model |>
  count(sex, .pred_class) |>
  group_by(sex) |>
  mutate(pct = n / sum(n))

ggplot(penguin_summary, aes(x = sex, y = pct, fill
= .pred_class)) +
  geom_col(position = "fill") +
  geom_text(aes(label = scales::percent(pct, accuracy = 1)),
            position = position_fill(vjust = 0.5)) +
  scale_y_continuous(labels = scales::percent)
```

Q: Percentages returning as just “1”

Cause: `count(sex)` only grouping by sex produced one row per group, so `n / sum(n)` trivially equaled 1.

Fix — include both grouping variables:

```
penguin_model |> count(sex, .pred_class)
```

Q: Just want overall sex proportions — female / (male + female)

No `group_by()` needed for a single-column proportion:

```
penguin_model |>
  count(sex) |>
  mutate(perc = n / sum(n))
```

Formatted as percent string:

```
penguin_model |>
  count(sex) |>
  mutate(perc = scales::percent(n / sum(n), accuracy = 1))
```

Single stacked bar version:

```
penguin_model |>
  count(sex) |>
  mutate(perc = n / sum(n)) |>
  ggplot(aes(x = "", y = perc, fill = sex)) +
```

```
geom_col() +  
geom_text(aes(label = scales::percent(perc, accuracy = 1)),  
           position = position_stack(vjust = 0.5)) +  
labs(x = NULL, y = "Percentage", fill = "Sex")
```

Q: Does `count()` change the shape of the data from that point on?

Yes. `count()` collapses row-level data into a summary table — it's shorthand for `group_by()` |> `summarise(n = n())`. All other columns are dropped, and any subsequent step in the pipe only sees the collapsed table.

Practical fix: save summaries to a new variable rather than overwriting the original:

```
sex_summary <- penguin_model |>  
  count(sex) |>  
  mutate(perc = n / sum(n))
```

Q: Why was `group_by()` throwing a wrench in the code?

Because `count(sex)` had already collapsed the data to one row per sex before `group_by(sex)` ran. Grouping a table that already has one row per group means `sum(n)` within each “group” is just that row's own value — so `n / sum(n)` always equals 1. `group_by()` is only useful when there are multiple rows per group left to sum across.

Q: `ggplot` broke after summarizing the data

`geom_bar()` counts rows itself — once `penguin_model` was collapsed to a summary table, there was nothing left to count. Swap to `geom_col()`, which plots pre-computed values:

```
ggplot(penguin_model, aes(x = sex, y = perc, fill = sex)) +  
  geom_col() +  
  geom_text(aes(label = scales::percent(perc, accuracy = 1)),  
            vjust = -0.5) +  
  scale_y_continuous(labels = scales::percent) +  
  labs(x = "Sex", y = "Percentage", title = "Proportion of  
Penguins by Sex")
```

Rule of thumb: `geom_bar()` = “count these rows for me”; `geom_col()` = “plot the height I already calculated.”

Q: Explain `geom_text(aes(label = scales::percent(perc, accuracy = 1)))`

- `geom_text()` — adds a text layer on top of an existing geometry (e.g., bars)
 - `aes(label = ...)` — maps the text shown per row/bar
 - `scales::percent(perc, accuracy = 1)` — formats a decimal as a percent string; `accuracy = 1` rounds to the nearest whole percent (smaller values like 0.1 or 0.01 give more decimal precision)
 - `vjust = -0.5` — nudges the label above the bar (for simple bar charts)
 - `position = position_stack(vjust = 0.5)` — centers the label within its segment of a stacked bar (regular `vjust` can't do this alone)
-

Q: Walk through null error rate

The error rate from always guessing the majority class — the baseline any real model should beat.

Null error rate = $1 - (\text{proportion of majority class})$

Example: with 39 female / 54 male (93 total), male is majority (58.1%):

Null error rate = $1 - 0.581 = 0.419$ ($\approx 41.9\%$)

Also called the “baseline error rate” or tied to “no-information rate” ($1 - \text{NIR}$).

Q: Easier way to do conditional column calculation than nested `ifelse()`

Use `dplyr::case_when()`:

```
penguin_model <- penguin_model |>
  mutate(category = case_when(
    perc > 0.5 ~ "majority",
    perc == 0.5 ~ "tied",
    perc < 0.5 ~ "minority",
```

```
      TRUE ~ NA_character_
    ))
```

- Conditions checked top-to-bottom, stops at first match
 - Always include a TRUE ~ ... catch-all
 - All right-hand outputs must be the same type (use NA_character_ for a typed NA)
 - Can reference multiple columns per condition
-

Q: case_when says column names not found

Common causes: 1. An earlier count()/summarise() already dropped the column (check with glimpse() mid-pipe) 2. Typo or case mismatch in column name 3. Referencing a column before it's created in the same mutate() (order matters — mutate evaluates top-to-bottom) 4. case_when() used outside of mutate() — it needs that context to attach to a data frame

Q: Missing pipe between two lines

```
penguin_model <- penguin_model
mutate(Threshold_Results = case_when(
  .pred_female > 0.8 ~ "TP"
))
```

Missing |> between the two lines — they're actually two disconnected statements, so mutate() never receives penguin_model and can't find .pred_female.

Fixed, plus fuller TP/FP/TN/FN logic (comparing prediction *and* ground truth):

```
penguin_model <- penguin_model |>
mutate(Threshold_Results = case_when(
  .pred_female > 0.8 & sex == "female" ~ "TP",
  .pred_female > 0.8 & sex == "male"   ~ "FP",
  .pred_female <= 0.8 & sex == "male" ~ "TN",
  .pred_female <= 0.8 & sex == "female" ~ "FN",
  TRUE ~ NA_character_
))
```

Q: Error — “must be a two-sided formula, not a logical vector”

Usually a missing comma between `case_when()` conditions — every line needs a trailing comma except the last.

Q: All NAs returned in `Threshold_Results`

Cause:

```
.pred_female > 0.8 & .pred_female == "female" & .pred_female ==  
sex ~ "TP"
```

`.pred_female` is numeric — it can never equal the string "female" or match `sex` directly, so the condition is always FALSE, and with no `TRUE ~ ...` fallback, every row becomes NA.

Fix — compare the threshold decision to the actual sex column, not to itself:

```
case_when(  
  .pred_female > 0.8 & sex == "female" ~ "TP",  
  .pred_female > 0.8 & sex == "male"   ~ "FP",  
  .pred_female <= 0.8 & sex == "male"  ~ "TN",  
  .pred_female <= 0.8 & sex == "female" ~ "FN",  
  TRUE ~ NA_character_  
)
```

Q: How does the probability threshold dictate TP/TN/FP/FN?

The threshold converts a continuous probability into a binary prediction:

```
if .pred_female > threshold → predict "female"  
if .pred_female ≤ threshold → predict "male"
```

Crossing that prediction against the actual sex:

	Actual: Female	Actual: Male
Predicted: Female	TP	FP
Predicted: Male	FN	TN

Raising the threshold makes the model stricter (fewer predicted positives → fewer FP, more FN). Lowering it makes the model looser (more predicted positives → more FP, fewer FN). This is the precision/recall tradeoff — there's no universally "correct" threshold, it depends on which error type is more costly.

Q: How to best present table results in a .qmd

- `knitr::kable()` — simplest, built-in, renders in HTML/PDF/Word:

```
penguin_model |> count(Threshold_Results) |>
knitr::kable(caption = "Threshold Classification Results")
```

- `gt` — more styling control (headers, formatting, conditional formatting):

```
library(gt)
penguin_model |> count(Threshold_Results) |> gt() |>
tab_header(title = "Threshold Classification Results")
```

- `DT::datatable()` — interactive, sortable, HTML output only

Quarto tips: use `#| tbl-cap:` chunk options for captions/cross-referencing, `#| echo: false` to hide code. For confusion-matrix-specific output, `yardstick::conf_mat() + autoplot()/summary()` computes metrics automatically.

Q: Pipeline from TP/TN/FP/FN count tables to accuracy/precision/recall/F1 without manual calculation

Option 1 — yardstick from raw predictions (most robust):

```
library(yardstick)

penguin_model |>
  mutate(pred_class_08 = if_else(.pred_female > 0.8, "female",
"male")) |>
  mutate(across(c(sex, pred_class_08), as.factor)) |>
  summarise(
    accuracy = accuracy_vec(sex, pred_class_08),
    precision = precision_vec(sex, pred_class_08),
    recall = recall_vec(sex, pred_class_08),
    f1 = f_meas_vec(sex, pred_class_08)
  )
```

Across multiple thresholds:

```

library(purrr)

compute_metrics <- function(threshold) {
  preds <- penguin_model |>
    mutate(pred_class = factor(if_else(.pred_female > threshold,
    "female", "male"))),
    sex = factor(sex))
  tibble(
    threshold = threshold,
    accuracy = accuracy_vec(preds$sex, preds$pred_class),
    precision = precision_vec(preds$sex, preds$pred_class),
    recall = recall_vec(preds$sex, preds$pred_class),
    f1 = f_meas_vec(preds$sex, preds$pred_class)
  )
}

metrics_table <- map_dfr(c(0.2, 0.5, 0.8), compute_metrics)

```

Option 2 — derive from existing TP/TN/FP/FN count tables:

```

calc_metrics <- function(tp, tn, fp, fn) {
  tibble(
    accuracy = (tp + tn) / (tp + tn + fp + fn),
    precision = tp / (tp + fp),
    recall = tp / (tp + fn),
    f1 = 2 * (precision * recall) / (precision + recall)
  )
}

your_table |>
  pivot_wider(names_from = Threshold_Results, values_from = n) |>
  mutate(calc_metrics(TP, TN, FP, FN))

```

Option 3 — caret::confusionMatrix():

```

library(caret)
cm <- confusionMatrix(pred_class, sex, positive = "female")
cm$byClass[c("Precision", "Recall", "F1")]
cm$overall["Accuracy"]

```

Q: How to join tables with the same columns

Same-column tables should be **stacked**, not joined — use `bind_rows()`:

```
combined_table <- bind_rows(table1, table2, table3)
```

Labeling source with `.id`:

```
combined_table <- bind_rows(
  threshold_02 = table1,
```



```
threshold_05 = table2,  
threshold_08 = table3,  
.id = "threshold"  
)
```

True joins (different columns, shared key) use `left_join()`, `inner_join()`, `full_join()`, etc. — not applicable here since these tables share the same structure.

Q: could not find function "calc_metrics" during render

Function existed in the console/session but not in the `.qmd` document itself, or its defining chunk: 1. Was never actually placed in a chunk (only run interactively) 2. Appears *after* the chunk that calls it (Quarto renders top-to-bottom in document order) 3. Has `eval: false` set, so it's displayed but never run 4. Has a typo mismatching the call

Fix: restart R and run chunks top-to-bottom as Quarto would, to catch exactly where it breaks.

Q: RStudio's Git tab disappeared

Caused by renaming the project's repository folder. RStudio's project/recent-projects references pointed to the old path, so it silently lost the Git/project association (the `.git` folder itself is untouched).

Fix: reopen the project via its `.Rproj` file directly (double-click it, or File → Open Project), rather than opening files individually or relying on auto-reopen.

Alternative fix: rename the folder back to its original name — this restores the exact path RStudio had stored, and both the project and Git tab should reconnect. (Git history and any GitHub remote are unaffected either way, since GitHub only tracks the remote URL, not the local folder name.)